Day 3 - Retrieving Data with SELECT

1. What SELECT Does

`SELECT` **reads** data. It never changes the table — it returns a temporary **result set** of rows and columns. Only `SELECT` and `FROM` are compulsory; everything else is optional.

```
SELECT   column_list
FROM     table_name
WHERE    condition
ORDER BY column_name
LIMIT    number;
```

Step	Clause	What it does
1	`FROM`	Picks up the table
2	`WHERE`	Removes unwanted rows
3	`SELECT`	Picks the columns
4	`ORDER BY`	Sorts the result
5	`LIMIT`	Keeps only a few rows

This order explains two rules: `WHERE` **cannot** use a column alias (`SELECT` has not run yet), but `ORDER BY` **can**.

2. Table Used in These Notes

The same `students` table is used for Days 3 to 6. Run it once.

```
CREATE DATABASE IF NOT EXISTS training;
USE training;
DROP TABLE IF EXISTS students;

CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name       VARCHAR(50) NOT NULL,
    city       VARCHAR(50),
    age        INT,
    course     VARCHAR(50),
    marks      INT,
    joined_on  DATE
);

INSERT INTO students VALUES
(101,'Rahul Verma','Hyderabad',21,'Python',78,'2025-01-15'),
(102,'Anita Sharma','Chennai',22,'SQL',95,'2025-01-20'),
(103,'Karan Patel','Hyderabad',20,'Python',38,'2025-02-01'),
(104,'Priya Nair','Kochi',23,'Java',66,'2025-02-10'),
(105,'Vikram Rao','Hyderabad',21,'SQL',81,'2025-03-05'),
(106,'Sneha Iyer','Chennai',22,'Java',54,'2025-03-12'),
(107,'Arjun Mehta','Pune',24,'DSA',90,'2025-04-02'),
(108,'Divya Menon','Kochi',20,'Python',45,'2025-04-18'),
(109,'Rohit Sinha','Pune',23,NULL,78,'2025-05-01'),
(110,'Meera Nair','Chennai',21,'DSA',NULL,'2025-05-20');
```

Two rows are deliberately incomplete: **Rohit Sinha** has no course, **Meera Nair** has no marks. They are used to explain `NULL`.

3. Choosing Columns and Rows

```
SELECT * FROM students;                    -- all columns
SELECT name, city FROM students;           -- only these, in the order you list them
SELECT DISTINCT city FROM students;        -- 10 students -> 4 different cities

SELECT name, marks FROM students WHERE marks > 70;        -- filter rows
SELECT name, city  FROM students WHERE city = 'Chennai';  -- text needs single quotes
```

Avoid `SELECT *` in real projects — you get columns you do not need, and the query changes silently when someone adds a column. `DISTINCT` applies to **all** selected columns together, so `SELECT DISTINCT city, age` returns unique *combinations*.

Operator	Meaning	Example
`=`	Equal to	`marks = 78`
`<>` or `!=`	Not equal to	`city <> 'Pune'`
`>` / `<`	Greater / less than	`marks > 70` · `age < 21`
`>=` / `<=`	Greater / less than or equal to	`marks >= 50` · `age <= 22`

Comparison uses a **single** `=`, never `==`. In MySQL text comparison is not case sensitive by default, but do not depend on it.

4. Combining Conditions

```
SELECT name, city, age FROM students
WHERE city = 'Hyderabad' AND age < 21;   -- AND: both true.  OR: any one true.
```

`AND` is evaluated **before** `OR` — when you mix them, always use brackets.

5. Sorting, Limiting and Skipping

```
SELECT name, marks FROM students ORDER BY marks DESC;              -- ASC is the default
SELECT name, city, marks FROM students ORDER BY city, marks DESC;   -- 2nd column breaks ties
SELECT name, marks FROM students ORDER BY marks DESC LIMIT 3;          -- top 3
SELECT name, marks FROM students ORDER BY marks DESC LIMIT 3 OFFSET 3;   -- ranks 4 to 6
```

- In `DESC` order a `NULL` comes **last** — MySQL treats `NULL` as the smallest value.
- Without `ORDER BY` no order is guaranteed, so `LIMIT` alone is meaningless — "top 3" needs a sort.
- `OFFSET` skips rows before returning any; this is how pagination works.

Two separate ranges at once (rows 3–7 and 11–12) cannot be done with `LIMIT` alone:

```
SELECT * FROM (                                -- Method 1: ROW_NUMBER, cleaner
    SELECT *, ROW_NUMBER() OVER (ORDER BY marks DESC) AS rn FROM students
) t
WHERE rn BETWEEN 3 AND 7 OR rn BETWEEN 11 AND 12;

SELECT * FROM students ORDER BY marks DESC LIMIT 5 OFFSET 2     -- Method 2: UNION ALL
UNION ALL
SELECT * FROM students ORDER BY marks DESC LIMIT 2 OFFSET 10;
```

6. Aliases and Calculated Columns

```
SELECT name AS student_name, marks AS score FROM students;
SELECT name, marks, marks + 5 AS bonus FROM students;       -- operators: + - * / %
```

An alias only changes the **heading**, and a calculated column exists only in the result — stored data is untouched (use `UPDATE` for that). `AS` is optional but keep it; for an alias containing a space use `AS "Student Name"`.

7. Working with NULL

`NULL` means **unknown** — not zero, not empty text. Any comparison with it is unknown, so `WHERE marks = NULL` returns an empty set with **no error message**.

```
SELECT name FROM students WHERE marks IS NULL;            -- Meera Nair
SELECT name, course FROM students WHERE course IS NOT NULL;
SELECT name, IFNULL(marks, 0) AS marks FROM students;     -- show 0 in place of NULL
```

`COALESCE(marks, 0)` does the same and works in every database; `IFNULL` is MySQL only.

8. Common Errors

Error	Cause	Fix
`1054` Unknown column 'nam' in 'field list'	Misspelt column	Check with `DESC students;`
`1146` Table 'training.student' doesn't exist	Wrong table name	`SHOW TABLES;` — it is `students`
`1054` Unknown column 'score' in 'where clause'	Alias used in `WHERE`	`WHERE` runs before `SELECT`; repeat the real column
`1054` Unknown column 'Chennai' in 'where clause'	Text without quotes	`WHERE city = 'Chennai'`
`1064` Syntax error near 'marks'	`ORDER marks`	`ORDER BY marks`
Empty set, no error at all	`WHERE marks = NULL`	Use `IS NULL`

9. Summary

Clause	Question it answers
`SELECT`	Which columns do I want?
`FROM`	Which table are they in?
`WHERE`	Which rows do I want?
`ORDER BY`	In what order?
`LIMIT` / `OFFSET`	How many rows, and from where?

Rules worth remembering: text needs **single quotes**, comparison uses a single `=`, `NULL` needs `IS NULL` and never `= NULL`, an alias works in `ORDER BY` but not in `WHERE`, and `LIMIT` is meaningless without `ORDER BY`. Also covered: `DISTINCT`, `AND` / `OR`, `AS`, `IS NOT NULL`, `IFNULL`.

10. Practice Questions

1. Display all columns, then only `name`, `course` and `marks`.
2. Display the distinct courses.
3. Display students whose marks are above 60, and students from `Kochi`.
4. Display students from `Hyderabad` whose age is under 21.
5. Sort all students by name ascending, then show the top 5 by marks.
6. Display students ranked 3rd to 5th by marks.
7. Display `name` headed `Student` and `marks` headed `Score`.
8. Add a column showing marks out of 200 (`marks * 2`).
9. Display students whose marks are missing, and those whose course is missing.
10. Display all students, showing `0` where marks are missing.
11. Explain why `WHERE marks = NULL` returns nothing.

Day 4 - Operators and Clauses

1. Why These Operators?

`=`, `>` and `<` are enough for simple conditions but become clumsy quickly. These operators make `WHERE` shorter **and** easier to read: `IN` replaces a long chain of `OR`, `BETWEEN` replaces two conditions joined by `AND`, `LIKE` searches for a **pattern** instead of an exact value, and `EXISTS` checks whether related rows exist in another table. All examples use the `students` table from Day 3.

2. IN and NOT IN

`IN` checks whether a value matches **any value in a list**; `NOT IN` matches **none** of them.

```
SELECT name, course FROM students WHERE course IN ('Python','SQL');
SELECT name, course FROM students WHERE course = 'Python' OR course = 'SQL';  -- same result
SELECT name, course FROM students WHERE course NOT IN ('Python','SQL');
```

`Rohit Sinha` is missing from the `NOT IN` result even though his course is neither — his course is `NULL`, and `NULL` can never be compared.

3. BETWEEN and NOT BETWEEN

`BETWEEN` checks a range and **includes both ends**. The **smaller value must come first** — `BETWEEN 22 AND 21` returns nothing.

```
SELECT name, age FROM students WHERE age BETWEEN 21 AND 22;   -- 21 and 22 both included
SELECT name, age FROM students WHERE age >= 21 AND age <= 22; -- the same thing
SELECT name, marks FROM students WHERE marks NOT BETWEEN 50 AND 90;   -- outside the range

SELECT name, joined_on FROM students                          -- its most common use: dates
WHERE joined_on BETWEEN '2025-01-01' AND '2025-02-28';
```

Dates are written as `'YYYY-MM-DD'`, inside single quotes.

4. LIKE — Pattern Matching

Two wildcards: `%` is any number of characters (including none), `_` is exactly one character.

```
SELECT name FROM students WHERE name LIKE 'R%';       -- starts with R
SELECT name FROM students WHERE name LIKE '%Nair';    -- ends with Nair
SELECT name FROM students WHERE name LIKE '%a%';      -- contains a
SELECT name FROM students WHERE name LIKE '_a%';      -- second letter is a: R-a-hul, K-a-ran
SELECT name FROM students WHERE name LIKE 'A%' OR name LIKE 'V%';   -- combines like any condition
```

Pattern	Meaning	Pattern	Meaning
`'R%'`	Starts with R	`'_a%'`	Second character is a
`'%a'`	Ends with a	`'__a%'`	Third character is a
`'%Nair%'`	Contains Nair anywhere	`'R%a'`	Starts with R and ends with a

In MySQL `LIKE` is not case sensitive by default, so `'r%'` also finds `Rahul`. Other databases differ.

5. AND, OR, NOT and Brackets

`AND` — both conditions true. `OR` — any one true. `NOT` — reverses a condition.

```
SELECT name, city, marks FROM students WHERE city = 'Hyderabad' AND marks > 60;
SELECT name, city FROM students WHERE city = 'Kochi' OR city = 'Pune';
SELECT name, marks FROM students WHERE NOT marks > 70;
```

`NOT marks > 70` drops `Meera Nair`: her marks are `NULL`, `NULL > 70` is unknown, and `NOT unknown` is still unknown.

`AND` is evaluated before `OR`. Wanted: *students from Hyderabad or Kochi who scored above 60.*

```
WHERE city = 'Hyderabad' OR city = 'Kochi' AND marks > 60;     -- WRONG
WHERE city = 'Hyderabad' OR (city = 'Kochi' AND marks > 60)    -- how MySQL read it
WHERE (city = 'Hyderabad' OR city = 'Kochi') AND marks > 60;   -- CORRECT
```

The wrong version returns `Karan Patel` with **38** marks, because the marks test was applied only to Kochi. There is **no error** — just a wrong answer. Whenever a query mixes `AND` and `OR`, always use brackets; even when the result is right, brackets show your intention.

6. EXISTS and NOT EXISTS

`EXISTS` is true when a subquery returns **at least one row**. It is used to test for related data in another table.

```
CREATE TABLE courses (course_id INT PRIMARY KEY, course_name VARCHAR(50), duration INT, fee INT);
INSERT INTO courses VALUES (1,'Python',45,15000), (2,'SQL',30,10000),
(3,'Java',60,20000), (4,'DSA',90,25000), (5,'Cloud',30,18000);

SELECT course_name FROM courses c                    -- courses somebody joined
WHERE EXISTS (SELECT 1 FROM students s WHERE s.course = c.course_name);

SELECT course_name FROM courses c                    -- courses nobody joined -> Cloud
WHERE NOT EXISTS (SELECT 1 FROM students s WHERE s.course = c.course_name);
```

`SELECT 1` is used because the values do not matter — `EXISTS` only checks **whether** rows come back.

Point	`IN`	`EXISTS`
Compares	A value against a list	Whether rows exist
Subquery returns	One column	Anything (`SELECT 1`)
Handles `NULL`	No — `NOT IN` breaks	Yes — safe
Better when	The list is small	The subquery is large

Simple rule: `IN` for a short list of values, `EXISTS` when checking another table.

7. Common Errors

Query	Result	Reason and fix
`course NOT IN ('Python','SQL',NULL)`	*Empty set*, no error	Becomes `… AND course <> NULL`, always unknown, so no row can pass. Use `NOT EXISTS`, or remove the `NULL`
`age BETWEEN 22 AND 21`	*Empty set*	Means `age >= 22 AND age <= 21`, impossible. Smaller value first
`course IN (Python, Java)`	`1054` Unknown column 'Python'	Missing quotes — MySQL reads it as a column name. Use `IN ('Python','Java')`
`course IN (Python, SQL)`	`1064` Syntax error near 'SQL)'	Same mistake, different error: `SQL` is a **reserved word**, so it cannot even be read as a column
`name = 'R%'`	*Empty set*	`=` compares the exact text `R%`; wildcards need `LIKE`

8. Summary

If you want to…	Use
Match one of several values	`IN` / `NOT IN`
Match a numeric or date range	`BETWEEN` / `NOT BETWEEN`
Search for part of a text	`LIKE` with `%` and `_`
Check another table for related rows	`EXISTS` / `NOT EXISTS`
Require both / accept either / reverse	`AND` / `OR` / `NOT`

Rules worth remembering: `BETWEEN` includes both ends and takes the smaller value first; `%` is many characters while `_` is exactly one; `AND` runs before `OR`, so **use brackets**; `NOT IN` fails silently when the list contains `NULL`, while `NOT EXISTS` is safe.

9. Practice Questions

1. Display students whose course is `Java` or `DSA` using `IN`, then rewrite it with `OR`.
2. Display students whose course is **not** `Python`.
3. Display students aged between 20 and 22, and those whose marks are **not** between 40 and 80.
4. Display students who joined between March and May.
5. Display students whose name starts with `A`, ends with `a`, and contains `Me`.
6. Display students whose name has `i` as the second letter.
7. Display students from Chennai or Pune who scored above 70 using brackets, then run it without brackets and explain the difference.
8. Display courses at least one student joined (`EXISTS`), and courses nobody joined (`NOT EXISTS`).
9. Explain why `NOT IN` with a `NULL` in the list returns nothing.

Day 5 - SQL Functions

1. Scalar vs Aggregate — the Main Idea

A **function** takes values, does something with them and returns a result. It never changes stored data, only the result. Functions work in `SELECT`, `WHERE`, `ORDER BY` and `GROUP BY`, and can be nested.

Type	Works on	Rows in → out	Examples
Scalar	One value at a time	10 → **10**	`UPPER`, `ROUND`, `LENGTH`
Aggregate	A whole column	10 → **1**	`COUNT`, `SUM`, `AVG`, `MAX`

All examples use the `students` table from Day 3.

2. String Functions

Function	Purpose	Function	Purpose
`UPPER(t)` / `LOWER(t)`	Capitals / small letters	`CONCAT(a,b,c)`	Joins text together
`LENGTH(t)`	Counts characters, spaces included	`REPLACE(t,old,new)`	Replaces **every** match
`SUBSTRING(t,start,n)`	`n` characters from `start`	`TRIM` / `LTRIM` / `RTRIM`	Removes spaces: both / left / right
`LEFT(t,n)` / `RIGHT(t,n)`	First / last `n` characters	`INSTR(t,x)`	Position of `x`, or `0` if absent

```
SELECT name, UPPER(name) AS caps, LENGTH(name) AS len,      -- Rahul Verma -> 11
       SUBSTRING(name,1,5) AS first5, LEFT(name,3) AS l3, RIGHT(name,4) AS r4,
       REPLACE(name,'a','@') AS replaced, INSTR(name,'a') AS pos_of_a
FROM students;

SELECT CONCAT(name,' - ',city) AS student_city FROM students;   -- Rahul Verma - Hyderabad
SELECT TRIM('   MySQL   ') AS trimmed, LTRIM('   Hi') AS lt, RTRIM('Hi   ') AS rt;
```

Text positions start at **1**, not 0. If any value inside `CONCAT` is `NULL`, the **whole result** becomes `NULL` — use `CONCAT_WS` or `IFNULL`. `TRIM` is mostly used when cleaning imported data; like every function it changes only the result, unless used inside an `UPDATE`.

3. Numeric Functions

Function	Purpose	Function	Purpose
`ROUND(n,d)`	Rounds to `d` decimals	`POWER(a,b)`	a to the power b
`CEIL(n)`	Rounds **up**	`SQRT(n)`	Square root
`FLOOR(n)`	Rounds **down**	`MOD(a,b)`	Remainder after division
`ABS(n)`	Removes the minus sign		

```
SELECT marks, ROUND(marks/3,2) AS rounded, CEIL(marks/3) AS up, FLOOR(marks/3) AS down
FROM students WHERE marks IS NOT NULL;              -- 95/3 -> 31.67, up 32, down 31
SELECT ABS(-45) AS absolute, POWER(2,5) AS power, SQRT(81) AS root, MOD(10,3) AS remainder;
SELECT 7/2 AS division, 7 DIV 2 AS integer_division, 7%2 AS remainder;   -- 3.5000 | 3 | 1
```

The three divisions differ: `/` gives `3.5000`, `DIV` the whole number `3`, `%` the remainder `1`. In PostgreSQL `7/2` gives `3`, because both numbers are integers.

4. Date Functions

Function	Returns	Function	Returns
`YEAR(d)` / `MONTH(d)` / `DAY(d)`	Year / month number / day	`DATEDIFF(later,earlier)`	Days between two dates
`MONTHNAME(d)` / `DAYNAME(d)`	Month name / day name	`DATE_ADD(d, INTERVAL n unit)`	Date after adding
`DATE_FORMAT(d,'fmt')`	Date as formatted **text**	`DATE_SUB(d, INTERVAL n unit)`	Date after subtracting
`STR_TO_DATE(t,'fmt')`	Text turned into a real date	`CURDATE()` / `NOW()`	Today / today with time
`LAST_DAY(d)`	Last date of that month		

```sql
SELECT name, YEAR(joined_on) AS yr, MONTH(joined_on) AS mth, DAY(joined_on) AS dy,
       MONTHNAME(joined_on) AS month_name, DAYNAME(joined_on) AS day_name FROM students;

SELECT DATE_FORMAT(joined_on,'%d-%m-%Y') AS formatted FROM students;   -- 15-01-2025
SELECT STR_TO_DATE('15-01-2025','%d-%m-%Y') AS converted_date;         -- back to a DATE
SELECT DATEDIFF('2025-06-01', joined_on) AS days_since FROM students;
SELECT DATE_ADD(joined_on, INTERVAL 30 DAY) AS after_30,
       DATE_SUB(joined_on, INTERVAL 1 MONTH) AS before_1m FROM students;
SELECT CURDATE() - INTERVAL 1 DAY AS yesterday, CURDATE() + INTERVAL 1 DAY AS tomorrow;
SELECT * FROM students WHERE joined_on >= CURDATE() - INTERVAL 30 DAY;
```

Code	`%Y`	`%y`	`%m`	`%M`	`%d`	`%W`
Means	4-digit year	2-digit year	Month number	Month name	Day number	Day name

Units for `DATE_ADD` / `DATE_SUB`: `DAY`, `WEEK`, `MONTH`, `QUARTER`, `YEAR`, `HOUR`, `MINUTE` — year changes and month lengths are handled for you, so one month before `2025-01-15` is `2024-12-15`. `DATE_FORMAT` returns **text**, not a date, so never sort or compare on its result: as text `'15-01-2025'` sorts before `'20-12-2024'`. The round trip is `DATE → DATE_FORMAT → text → STR_TO_DATE → DATE`. Prefer a **fixed date** over `CURDATE()` in exercises, so the answer does not change from day to day.

5. Aggregate Functions

Function	Returns	Ignores `NULL` ?
`COUNT(*)`	Number of **rows**	No
`COUNT(col)`	Number of **non-null values**	Yes
`SUM` / `AVG` / `MIN` / `MAX`	Total / average / smallest / largest	Yes

```sql
SELECT COUNT(*) AS total_rows, COUNT(marks) AS with_marks, SUM(marks) AS total,
       ROUND(AVG(marks),2) AS average, MIN(marks) AS lowest, MAX(marks) AS highest
FROM students;
-- 10 | 9 | 625 | 69.44 | 38 | 95      <- ten rows collapsed into one
```

`COUNT(*)` is **10** but `COUNT(marks)` is **9**, because one student has no marks. That also explains the average: `625 / 9 = 69.44`, not `625 / 10`. If a missing mark should count as zero you must say so:

```sql
SELECT ROUND(AVG(IFNULL(marks,0)),2) AS average_with_zeros FROM students;   -- 62.50
```

Two different answers, both correct — which one is right depends on the question. This is a very common interview question.

6. Conditional Functions

```sql
SELECT name, marks,
       CASE WHEN marks >= 75   THEN 'Distinction'
            WHEN marks >= 50   THEN 'Pass'
            WHEN marks IS NULL THEN 'Not graded'
            ELSE 'Fail'
       END AS result
FROM students;

SELECT name, IF(marks >= 50,'Pass','Fail') AS status FROM students;
SELECT name, IFNULL(marks,0) AS marks, COALESCE(course,'Not assigned') AS course
FROM students WHERE marks IS NULL OR course IS NULL;
```

- `CASE` conditions are checked **top to bottom** and the **first true one wins**, so put the strictest first — if `marks >= 50` came before `marks >= 75`, nobody would ever get a Distinction. Without `ELSE`, unmatched rows become `NULL`.
- `IF` is for two outcomes, `CASE` for three or more. `IF` and `IFNULL` are MySQL only; `CASE` and `COALESCE` are standard SQL, and `COALESCE` returns the first non-`NULL` of many values.

7. Common Errors

Query	Result	Reason and fix
`SELECT name, AVG(marks) FROM students;`	`1140` nonaggregated column	`AVG` returns one value but `name` has ten. Use `GROUP BY` (Day 6)
`WHERE COUNT(*) > 2`	`1111` Invalid use of group function	`WHERE` runs before grouping. Use `HAVING` (Day 6)
`SUBSTRING('Rahul',0,3)`	Empty text, no error	Positions start at **1**: `SUBSTRING('Rahul',1,3)` gives `Rah`
`CONCAT(name,' - ',course)` where course is `NULL`	`NULL`	One `NULL` poisons `CONCAT`: `CONCAT(name,' - ',IFNULL(course,'None'))`

8. Summary

Category	Functions
String	`UPPER` , `LOWER` , `LENGTH` , `SUBSTRING` , `LEFT` , `RIGHT` , `CONCAT` , `REPLACE` , `TRIM` , `LTRIM` , `RTRIM` , `INSTR`
Numeric	`ROUND` , `CEIL` , `FLOOR` , `ABS` , `POWER` , `SQRT` , `MOD` , `DIV`
Date	`YEAR` , `MONTH` , `DAY` , `MONTHNAME` , `DAYNAME` , `DATE_FORMAT` , `STR_TO_DATE` , `DATEDIFF` , `DATE_ADD` , `DATE_SUB` , `CURDATE` , `NOW` , `LAST_DAY`
Aggregate	`COUNT` , `SUM` , `AVG` , `MIN` , `MAX`
Conditional	`CASE` , `IF` , `IFNULL` , `COALESCE`

Rules worth remembering: scalar functions keep the row count while aggregates reduce all rows to one; text positions start at **1**; `COUNT(*)` counts rows and every other aggregate **ignores** `NULL` ; `AVG` divides by the non-null count; `CASE` takes the first true condition; one `NULL` inside `CONCAT` makes the whole result `NULL` ; aggregates cannot be used in `WHERE` or mixed with plain columns without `GROUP BY` .

9. Practice Questions

1. Display every name in capitals with its length, and the first 4 characters of each city.
2. Join name and course into one column reading `Rahul Verma (Python)` .
3. Replace every space in the name with an underscore.
4. Display marks divided by 7 rounded to 2 decimals, and the remainder when divided by 10.
5. Display the year and month name each student joined.
6. Display `joined_on` in `dd/mm/yyyy` format, then convert that text back into a date.
7. How many days passed between each joining date and `2025-06-01` , and what is the date 45 days after each student joined?
8. Show the total, average, highest and lowest marks in one query.
9. Count the rows and count the students who have marks; explain the difference, then recalculate the average treating missing marks as 0.
10. Label each student `Distinction` , `Pass` , `Fail` or `Not graded` using `CASE` .
11. Use `IF` to show `Senior` for age above 22, otherwise `Junior` , and `Not assigned` wherever a course is missing.

Day 6 - Grouping Data with GROUP BY and HAVING

1. Why Grouping?

An aggregate on its own collapses the **whole table** into one row — `SELECT AVG(marks) FROM students;` answers "*what is the class average?*" but not "*what is the average in each city?*". `GROUP BY` splits the rows into groups and applies the aggregate **separately inside each group**.

```
   10 students                          4 cities
   | Hyderabad  78 |                  | Hyderabad  65.67 |
   | Chennai    95 |    GROUP BY      | Chennai    74.50 |
   | Hyderabad  38 |   ----------->   | Kochi      55.50 |
   | Kochi      66 |      city        | Pune       84.00 |
   | ...           |
```

One row comes out **per group**, and grouping by more columns creates more, smaller groups. `GROUP BY` always comes **after** `WHERE`. All examples use the `students` table from Day 3.

2. Basic Grouping

```sql
SELECT city, COUNT(*) AS students FROM students GROUP BY city;
-- Hyderabad 3 | Chennai 3 | Kochi 2 | Pune 2      <- ten students became four rows

SELECT city, COUNT(*) AS students, ROUND(AVG(marks),2) AS avg_marks
FROM students GROUP BY city ORDER BY avg_marks DESC;
-- Pune 2 84.00 | Chennai 3 74.50 | Hyderabad 3 65.67 | Kochi 2 55.50
```

Chennai shows **3 students** but its average comes from only **2** — `Meera Nair` has no marks and `AVG` ignores `NULL`. `GROUP BY` does not sort; add `ORDER BY` when the order matters.

3. The Golden Rule

> Every column in `SELECT` must be either **inside an aggregate** or **listed in the** `GROUP BY`.

`SELECT city, name, COUNT(*) … GROUP BY city` fails with `1055 … not in GROUP BY clause`: Hyderabad has three students, so which single `name` should its one row show? The question has no answer, so MySQL refuses.

4. Grouping and NULL

`GROUP BY` puts **all** `NULL` **values into one group** — unlike `=`, which never matches `NULL`.

```sql
SELECT course, COUNT(*) AS students FROM students GROUP BY course ORDER BY course;
-- NULL 1 | DSA 2 | Java 2 | Python 3 | SQL 2      <- Rohit Sinha forms his own NULL group
```

5. Calculations and Multiple Columns

Aggregates are ordinary expressions, so you can do arithmetic with them.

```sql
SELECT city, COUNT(*) AS n, MIN(marks) AS lowest, MAX(marks) AS highest,
       MAX(marks) - MIN(marks) AS spread
FROM students WHERE marks IS NOT NULL GROUP BY city ORDER BY spread DESC;
-- Hyderabad 43 | Chennai 41 | Kochi 21 | Pune 12

SELECT city, age, COUNT(*) AS students FROM students GROUP BY city, age ORDER BY city, age;
-- one row per combination: four cities became eight groups
```

Spread shows how consistent a group is — Pune's students are close together, Hyderabad's are far apart. The average alone hides that.

6. HAVING — Filtering Groups

`HAVING` filters **groups** the same way `WHERE` filters **rows**.

```sql
SELECT city, COUNT(*) AS students FROM students
GROUP BY city HAVING COUNT(*) > 2;              -- Hyderabad 3, Chennai 3

SELECT city, ROUND(AVG(marks),1) AS avg_marks FROM students
GROUP BY city HAVING AVG(marks) > 70 ORDER BY avg_marks DESC;   -- Pune 84.0, Chennai 74.5
```

Point	`WHERE`	`HAVING`
Filters	Individual **rows**	Whole **groups**
Runs	**Before** grouping	**After** grouping
Aggregates allowed	No	Yes
Needs `GROUP BY`	No	Almost always

`SELECT city, COUNT(*) FROM students WHERE COUNT(*) > 2 GROUP BY city;` fails with `1111 Invalid use of group function` — `WHERE` runs before the groups exist, so `COUNT(*)` has no meaning yet.

7. Using WHERE and HAVING Together

They work at different stages, so a query often needs both. *Among students who passed, show the average per city, for cities with at least two such students:*

```
SELECT city, COUNT(*) AS passed, ROUND(AVG(marks),2) AS avg_marks
FROM students
WHERE marks >= 50          -- 1. keep only passing students
GROUP BY city              -- 2. group what is left
HAVING COUNT(*) >= 2       -- 3. keep cities with two or more of them
ORDER BY avg_marks DESC;
-- Pune 2 84.00 | Hyderabad 2 79.50 | Chennai 2 74.50
```

Hyderabad is **79.50** here but **65.67** in section 2 — `Karan Patel`'s 38 was removed by `WHERE` **before** the average was calculated. **The order of operations changes the answer**, which is why `WHERE` and `HAVING` are not interchangeable.

8. The Full Run Order

Step	1	2	3	4	5	6	7
Clause	`FROM`	`WHERE`	`GROUP BY`	`HAVING`	`SELECT`	`ORDER BY`	`LIMIT`
Does	Fetch table	Drop **rows**	Form groups	Drop **groups**	Work out columns	Sort	Cut

This explains three rules: `WHERE` cannot use an aggregate (they do not exist until step 3), `HAVING` can (step 4 is after grouping), and `ORDER BY` can use a `SELECT` alias (6 is after 5) while `WHERE` cannot (2 is before 5).

9. Two Classic Patterns

Finding duplicates — `GROUP BY` with `HAVING COUNT(*) > 1`. This appears in almost every SQL interview:

```
SELECT course, COUNT(*) AS n FROM students GROUP BY course HAVING COUNT(*) > 1 ORDER BY n DESC;
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;   -- same shape, real table
```

Grouping by a calculated value — you can group by a `CASE` expression, using its alias:

```
SELECT CASE WHEN marks >= 75   THEN 'Distinction'
            WHEN marks >= 50   THEN 'Pass'
            WHEN marks IS NULL THEN 'Not graded'
            ELSE 'Fail' END AS band,
       COUNT(*) AS students
FROM students GROUP BY band ORDER BY students DESC, band;
-- Distinction 5 | Fail 2 | Pass 2 | Not graded 1
```

`Pass` and `Fail` are tied on 2, so the second sort key makes the order predictable.

10. WITH ROLLUP — a Total Row

```
SELECT city, COUNT(*) AS n FROM students GROUP BY city WITH ROLLUP;
-- Chennai 3 | Hyderabad 3 | Kochi 2 | Pune 2 | NULL 10      <- the last row is the total
```

The total row is marked by `NULL` in the grouped column, which is confusing when that column also contains real `NULL`s — use `GROUPING()` to tell them apart.

11. Common Errors

Query	Result	Reason and fix
`SELECT city, name, COUNT(*) … GROUP BY city`	`1055` nonaggregated column	Remove the column, group by it, or wrap it in an aggregate
`WHERE COUNT(*) > 2 … GROUP BY city`	`1111` Invalid use of group function	Move the condition into `HAVING`
`GROUP BY city HAVING city = 'Pune'`	Works, but wrong practice	It groups every city then throws them away — filter rows early with `WHERE`
`COUNT(*)` vs `COUNT(marks)`	`10` and `9`, no error	`COUNT(*)` counts rows; `COUNT(column)` skips `NULL`

12. Summary

If you want to…	Use
Summarise the whole table	An aggregate alone
Summarise per group	`GROUP BY`
Remove unwanted rows first	`WHERE`
Remove unwanted groups after	`HAVING`
Find repeated values	`GROUP BY … HAVING COUNT(*) > 1`
Add a grand-total row	`WITH ROLLUP`

Rules worth remembering: every selected column must be **grouped or aggregated**; `WHERE` filters rows **before** grouping and `HAVING` filters groups **after**; only `HAVING` may contain an aggregate; all `NULL`s form a **single** group; and filtering with `WHERE` first can change the aggregate result, as Hyderabad's average showed.

13. Practice Questions

1. Count the students in each city, and in each course.
2. Show the average marks per city, highest average first.
3. Show the highest, lowest and spread of marks in each city.
4. Which group does `Rohit Sinha` fall into, and why?
5. Show the number of students per age, then count students by city **and** age.
6. Show cities with more than two students.
7. Show cities whose average marks are above 70.
8. Among students scoring 50 or more, show the average per city for cities with at least two such students.
9. Explain why Hyderabad's average changes between questions 2 and 8.
10. Find every course taken by more than one student.
11. Count students in each grade band using `CASE`.
12. Add a grand-total row to question 1 using `WITH ROLLUP`.
13. Why does `WHERE COUNT(*) > 2` fail but `HAVING COUNT(*) > 2` work?